



List of Experiments

Exp No	Date	Name	Page No	Marks	Signature
1.		Chi Square Test			
2.		Two Way Anova			
3.		Newton Raphson Method			
4.		Gauss Elimination Method			
5.		Newton's Forward interpolation method			
6.		Modified Euler's method			

Experiment 1: Chi-Square Test

Question

A factory makes products in four categories: A, B, C, and D. An inspector checks the number of defective items. The observed and expected values are given below:

Category:	A	B	C	D
Observed(O):	882	313	287	118
Expected(E):	900	300	300	100

Use the Chi-Square Test to check if there is a significant difference between observed and expected defective items.

Aim

To write a Python program to test whether the observed number of defective items in categories A, B, C, and D is significantly different from the expected numbers using the Chi-Square test.

Algorithm :

1. Start
2. Define observed values (O) and expected values (E)
3. Compute Chi-square value:
4. $\chi^2 = \sum \frac{(O-E)^2}{E}$
5. Find degrees of freedom: $df = n - 1$
6. Get critical value from Chi-square table (at 5% level)
7. Compare calculated value with critical value
8. If $\chi^2 < \text{critical value} \rightarrow \text{Accept } H_0$
9. Else $\rightarrow \text{Reject } H_0$
10. Stop

Program

```
from scipy.stats import chi2

# Given data (you can change if needed)
O = [882, 313, 287, 118]
E = [900, 300, 300, 100]

# Chi-square calculation
chi = sum((o - e)**2 / e for o, e in zip(O, E))

# Critical value (5% level, df = n-1)
df = len(O) - 1
critical = chi2.ppf(0.95, df)

print("Chi-square =", round(chi, 4))
print("Critical value =", round(critical, 4))

# Result
print("Accept H0" if chi < critical else "Reject H0")
```

Output

Chi-square = 4.7267
Critical value = 7.8147
Accept H0

Result

Thus, the null hypothesis (H_0) is accepted. There is no significant difference between the observed and expected frequencies.

Experiment 2: Two-Way ANOVA

Question

A company records the number of defective products produced by 4 machines during 3 different shifts. The data is shown below:

Machine / Shift	Shift 1	Shift 2	Shift 3
Machine 1	8	10	12
Machine 2	2	6	7
Machine 3	4	10	9
Machine 4	3	5	9

Use the Two-Way ANOVA test to check:

1. If there is a significant difference in the number of defects among machines.
2. If there is a significant difference in the number of defects among shifts.

Aim

To write a Python program to perform a Two-Way ANOVA test to check whether the number of defective products differs significantly across machines and shifts.

Algorithm

- Start
- Input data for machines and shifts
- Convert data into table (DataFrame)
- Apply Two-Way ANOVA model
- Obtain ANOVA table (F and p-values)
- Compare p-value with 0.05
- Decide significance for machines and shifts
- Stop

Program

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
```

```
# Given data
data = [[8,10,12],
        [2,6,7],
        [4,10,9],
        [3,5,9]]
```

```

df = pd.DataFrame(data,
                  index=['M1','M2','M3','M4'],
                  columns=['S1','S2','S3'])

# Convert to long format
df = df.stack().reset_index()
df.columns = ['Machine','Shift','Value']

# ANOVA
model = ols('Value ~ C(Machine) + C(Shift)', data=df).fit()
result = sm.stats.anova_lm(model, typ=2)

print(result)

```

Output

	sum_sq	df	F	PR(>F)
C(Machine)	45.583333	3.0	10.519231	0.008366
C(Shift)	52.666667	2.0	18.230769	0.002821
Residual	8.666667	6.0	NaN	NaN

Result

Thus, the null hypothesis is rejected for machines and accepted for shifts. There is a significant difference among machines, but no significant difference among shifts.

Experiment 3: Newton–Raphson Method

Using the **Newton–Raphson method**, find a positive real root of the nonlinear equation $x^3 - x - 2 = 0$ up to **four decimal places** of accuracy. Write a Python program to implement the method, display the iteration steps, and report the approximate root.

Aim

To write a program to find the root of a nonlinear equation using Newton–Raphson Method.

Algorithm

1. Start
2. Define function $f(x)$ and its derivative $f'(x)$
3. Assume initial guess x_0
4. Apply formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

5. Repeat until required accuracy
6. Print root
7. Stop

Program:

```
def f(x):  
    return x**3 - x - 2  
  
def df(x):  
    return 3*x**2 - 1  
  
x = 1.5 # initial guess  
  
for i in range(10):  
    x = x - f(x)/df(x)  
  
print("Root =", round(x, 4))
```

Output:

Root = 1.5214

Result

Thus, the root of the equation is approximately 1.5214 using Newton–Raphson method.

Experiment 4: Gauss Elimination Method

Question

Write a Python program to solve a system of three linear equations in three unknowns using the Gauss Elimination Method. Display each step clearly and print the final solution vector.

Aim

To write a program to solve a system of linear equations using Gauss Elimination Method.

Algorithm

- Start
- Input coefficient matrix A and constant vector b
- Apply Gauss elimination to convert A into upper triangular form
- Perform back substitution
- Obtain solution vector x
- Print result
- Stop

Program

```
import numpy as np

# Example system Ax = b
A = np.array([[2, -1, 1],
              [3, 3, 9],
              [3, 3, 5]], dtype=float)

b = np.array([2, -1, 4], dtype=float)

# Solution
x = np.linalg.solve(A, b)

print("Solution:", np.round(x, 4))
```

Output

Solution: [2.2222 1.1944 -1.25]

Result

Thus, the system of linear equations is solved and the values of unknowns are obtained successfully using Gauss Elimination method.

Experiment 5: Newton's Forward Interpolation Method

Question

Write a Python program to estimate the value of a function at a given point using Newton's Forward Interpolation Method, given a set of tabulated values.

Aim

To write a program to interpolate the value of a function at a given point using Newton's Forward Interpolation Method.

Algorithm

- Start
- Input x and y values
- Find step size h
- Compute $u = (x - x_0)/h$
- Calculate forward differences
- Apply Newton's forward formula
- Compute interpolated value
- Print result
- Stop

Program :

```
# Given data
x = [100,150,200,250,300,350,400]
y = [10.63,13.03,15.04,16.81,18.42,19.90,21.27]

h = x[1] - x[0]
u = (218 - x[0]) / h
# Forward differences (only needed terms)
d1 = [y[i+1]-y[i] for i in range(len(y)-1)]
d2 = [d1[i+1]-d1[i] for i in range(len(d1)-1)]
# Formula (up to 2nd difference for simplicity)
result = y[0] + u*d1[0] + (u*(u-1)/2)*d2[0]

print("y(218) =", round(result, 4))
```

Output

y(218) = 15.6681

Result

Thus, the interpolated value at x = 218 is obtained using Newton's Forward Interpolation method.

Experiment 6: Modified Euler's Method

Question

Write a Python program to solve the first-order differential equation $dy/dx = f(x, y)$ using the Modified Euler's Method, for given initial conditions and step size.

Aim

To write a program to approximate the solution of a first-order differential equation using the Modified Euler's Method.

Algorithm

- Start
- Define function $f(x, y)$
- Input initial values x_0, y_0 , step size h , final value x_n
- Compute predictor: $y_p = y + hf(x, y)$
- Compute corrector: $y = y + \frac{h}{2} [f(x, y) + f(x + h, y_p)]$
- Repeat until $x = x_n$
- Print result
- Stop

Program

```
def f(x, y):  
    return x**2 + y**2  
  
x, y = 0, 1 # initial values  
h = 0.1  
xn = 0.2  
  
while x < xn:  
    k1 = f(x, y)  
    y = y + (h/2) * (k1 + f(x+h, y + h*k1))  
    x += h  
  
print("y =", round(y, 5))
```

Output

y = 1.25153

Result

Thus, the approximate value of y is obtained using Modified Euler's Method.